New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6923: Written Homework 4.
Due Friday, November 21, 2025, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. Copying from GPTs or other LMs is not allowed.

Problem 1. Randomized Coordinate Descent (8pts)

Randomized coordinate descent (RCD) is a close relative of stochastic gradient descent that is often used for minimizing loss functions in machine learning. See below for pseudocode. Note that for a bolded vector $\mathbf{a}$, we used a_i to denote the i^{th} entry.

- Choose starting vector $\boldsymbol{\beta} \in \mathbb{R}^d$ and positive learning rate η.
- For $i = 1, \ldots, T$
  - Compute $\mathbf{g} = \nabla L(\boldsymbol{\beta})$.
  - Choose j uniformly at random from $j \in \{1, \ldots, d\}$.
  - For index j, update $\beta_j \leftarrow \beta_j - \eta \cdot g_j$.

The method is similar to stochastic gradient descent in that it often allows for cheaper per-iteration cost than gradient descent. It does so by updating one randomly chosen entry (the j^{th} entry) of the parameter vector $\boldsymbol{\beta}$. The vector is updated by subtracting off a scaling of the j^{th} entry of the current gradient.

(a) Let $\boldsymbol{\beta}^{(i)}$ be the value of the parameter vector $\boldsymbol{\beta}$ at the end of the i^{th} iteration of the for loop above. Prove that $\mathbb{E}[\boldsymbol{\beta}^{(i)} - \boldsymbol{\beta}^{(i-1)}] = -c\nabla L(\boldsymbol{\beta}^{(i-1)})$ for some positive scalar constant c. I.e., like SGD, RCD moves in the direction of the negative gradient in expectation.

(b) Prove that, like the steepest descent methods considered in Problem 2 of Homework 3 $\lim_{\eta \to 0} \frac{L(\boldsymbol{\beta}^{(i-1)}) - L(\boldsymbol{\beta}^{(i)})}{\eta}$ is positive for stochastic coordinate descent. Interestingly, this same claim is *not true* for stochastic gradient descent. Even as the step size goes to zero, an SGD update is not guaranteed to decrease the objective value.

(c) **Optional Bonus (5pts).** Consider the least squares regression loss, $L(\boldsymbol{\beta}) = \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2$ for a data matrix $\mathbf{X}$ with d columns. Recall that this loss has gradient $\nabla L(\boldsymbol{\beta}) = 2\mathbf{X}^T(\mathbf{X}\boldsymbol{\beta} - \mathbf{y})$. Show that, after an upfront cost of $O(nd)$ on the first iteration, each subsequent iteration of coordinate descent on this loss can be implemented in $O(n)$ time. Write pseudocode showing an $O(n)$ time implementation. **Hint:** This problem requires some cleverness. Try to use the information from previous iterations to your advantage – you cannot be able to compute $\mathbf{g}$ from scratch at each iteration.

Problem 2: Sample Complexity for Infinite Hypothesis Classes (8pts)

In the lecture on learning theory, we studied how to bound the number of training examples required to PAC-learn certain functions (also called models or hypothesis classes) in the realizable setting. We focused mainly on *finite* hypothesis classes and showed that $m = O(\log |\mathcal{H}|)$ samples are sufficient (up to constants and dependence on ε, δ) to ensure that, with high probability $1 - \delta$, an ERM learner will output a hypothesis whose true error is at most ε. In this homework problem, we will extend beyond finite classes and introduce the Vapnik–Chervonenkis (VC) dimension.

We say a hypothesis class $\mathcal{H}$ *shatters* a set of points $\mathbf{x}_1, \ldots, \mathbf{x}_q \in \mathbb{R}^d$ if there is some hypothesis $h \in \mathcal{H}$ that matches *any possible labeling* of the data. The VC dimension of a hypothesis class $\mathcal{H}$ over points in $\mathbb{R}^d$ is the size of the *largest* point set that $\mathcal{H}$ shatters.

Vapnik and Chervonenkis in 1971 proved that, for a hypothesis class $\mathcal{H}$ with VC dimension V, $\frac{2\log(1/\epsilon)}{\epsilon}\left(V + \log\frac{2}{\delta}\right)$ samples suffices to guarantee PAC-learnability.

1. It is known that the VC dimension of the set of linear classifiers in $\mathbb{R}^d$ is $d+1$. Let's understand this for $d = 2$. Provide an example that shows VC-dimension of the set of linear classifiers in $\mathbb{R}^2$ is less than 4? (Use the example provided in Lecture 7 slide 63)

2. Provide an example that shows VC-dimension of the set of linear classifiers in $\mathbb{R}^2$ is at least 3? (Use the example provided in Lecture 7 slide 63)

3. Parity of a Boolean vector $x \in \{0,1\}^n$ is 1 if it contains an odd number of 1's, and 0 otherwise. Consider the hypothesis class of all parity functions over subsets of coordinates, represented by vectors $w \in \{0,1\}^n$:

$$\mathcal{H}_{\text{parity}} = \left\{ h_w : \{0,1\}^n \to \{0,1\} \;\middle|\; w \in \{0,1\}^n,\ h_w(x) = \bigoplus_{i=1}^{n} (w_i x_i) \right\},$$

where $\oplus$ denotes the XOR. First show that the VC dimension of $\mathcal{H}_{\text{parity}}$ is at least n. Then show that VC dimension of $\mathcal{H}_{\text{parity}}$ is at most n.

Problem 3: Kernels for Shifted Images (15pts)

In class we discussed why the Gaussian kernel is a better similarity metric for MNIST digits than the inner product. Here we consider an additional modification to the Gaussian kernel that is very similar to state-of-the-art kernels for digit and character classification, which can achieve 99%+ accuracy on MNIST.

For illustration purposes we consider 5x5 black and white images: a pixel has value 1 if it is white and value 0 if it is black. For example, consider the following images of two 0s and two 1s:

$$I_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad I_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad I_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad I_4 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(a) Let $\mathbf{x}_i \in \{0,1\}^{25}$ denote the vectorized version of image I_i, obtained by concatenating the rows of the matrix representation of the image into a vector. Compute the 4×4 kernel matrix $\mathbf{K}$ for images $I_1, \ldots, I_4$ using the standard Gaussian kernel $k_G(I_i, I_j) = e^{-\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}$ (this is just a simple calculation, but you might want to do it in Python to save time).

(b) Suppose I_1 and I_2 are in our training data and I_3 and I_4 are in our test data. Which training image is most similar to each of our test images according to Gaussian kernel similarity? Do you expect a kernel classifier (e.g., a 1-nearest neighbor method) to correctly or incorrectly classify I_3 and I_4?

(c) Consider a "left-right shift" kernel, which is a similarity measure defined as follows:

For an image I_i, let I_i^{right} be the image with its far right column removed and let I_i^{left} be the image with its far left column removed. Intuitively, I_i^{right} corresponds to the image shifted one pixel to the right and I_i^{left} corresponds to the image shifted one pixel left. Define a new similarity metric k_{shift} as follows:

$$k_{\text{shift}}(I_i, I_j) = k_G(I_i^{\text{right}}, I_j^{\text{right}}) + k_G(I_i^{\text{left}}, I_j^{\text{left}}) + k_G(I_i^{\text{right}}, I_j^{\text{left}}) + k_G(I_i^{\text{left}}, I_j^{\text{right}})$$

Intuitively this kernel captures similarity between images which are similar *after a shift*, something the standard Gaussian kernel does not account for.

Recompute the a 4×4 kernel matrix $\mathbf{K}$ for images $I_1, \ldots, I_4$ using k_{shift}.

(d) Again I_1 and I_2 were in our training data and I_3 and I_4 were in our test data. Now which training image is most similar to each of our test images according to the "left-right shift" kernel? Do you expect a kernel classifier (e.g., a 1-nearest neighbor method) to correctly or incorrectly classify I_3 and I_4?

(e) Prove that k_{shift} is a positive semi-definite kernel function. **Hint:** Use the fact that k_G is positive semi-definite.

Problem 4: Backprop Linear Algebra (8pts)

We saw in the class how the backpropagation algorithm works for computing the partial derivatives with respect to all parameters. Backpropagation can be used to compute derivatives with respect to one particular input in $O(d)$ time. There however is another appealing aspect to backpropagation algorithm: the final computation boils down to linear algebra operations (matrix multiplication and vector operations) which can be performed quickly on a GPU. In this problem we verify this. Let's fix some notations:

- Let $\mathbf{v}_i$ be a vector containing the value of all nodes j in layer i.
- Let $\bar{\mathbf{v}}_i$ be a vector containing $\bar{j}$ for all nodes j in layer i.
- Let $\boldsymbol{\delta}_i$ be a vector containing $\partial z/\partial j$ for all nodes j in layer i.
- Let $\bar{\boldsymbol{\delta}}_i$ be a vector containing $\partial z/\partial \bar{j}$ for all nodes j in layer i.
- Let $\mathbf{W}_i$ be a matrix containing all the weights for edges between layer i and layer $i+1$.

1. Using the notation above, show node derivative computation is a matrix multiplication. That is write down $\boldsymbol{\delta}_i$ in terms of a matrix multiplication and justify your answer.
2. Let $\boldsymbol{\Delta}_i$ be a matrix contain the derivatives for all weights for edges between layer i and layer $i+1$. Using the notation above, show that $\boldsymbol{\Delta}_i$ is equal to an outer-product.